



LYCANS AERODESIGN TEAM

Trainee Assignment Report

Name: Trainee Member

Assignment: Final Project

Date: September 13, 2026

1 Overall System Architecture and Data Flow

1.1 System Overview

A smartphone camera streams live video via IP Webcam to OpenCV for object detection. The **Perception Node** processes this and sends custom state frames to the **Decision Node**, which directs ArduPilot SITL maneuvers via MAVROS.

Pipeline: Camera → OpenCV Processing → Custom ROS 2 Message → Decision Node → MAVROS
→ ArduPilot SITL

1.2 Object-Oriented Design

We chose an Object-Oriented Programming (OOP) software architecture because it provides a highly structured and organized paradigm for team-based development. The codebase is modularized as follows:

- **cv_perception.py:**
 - **cameraStream:** Uses `get_frame()` to return an active frame on each loop execution.
 - **TargetDetector:** Applies `detect_blue_rectangle()` and `detect_green_circle()` for color masking and shape classification, returning command integers: NONE (0), RTL (1), or LAND (2).
 - **ObstacleDetector:** Uses `detect_red_obj()` for masking. Computes object-to-frame area ratio to classify distance ranges (TOOFAR, FAR, MID, NEAR). Uses `obstacle_direction()` to determine the X-coordinate column (LEFT, CENTER, RIGHT).
- **perception_node.py:**
 - **PackMsg & PerceptionNode:** Uses `packvars()` to pack output values into a ROS 2 custom `Detection.msg` and publishes it onto the `vision/detection_cmd` topic.
- **decision_manager.py:**
 - **DetectionInput & Decision:** Encapsulates raw data (command, obstacle presence, distance) and chosen flight actions.
 - **DecisionManager:** Evaluates `DetectionInput`. Returns avoidance decisions for hazards, or RTL/LAND decisions for targets. Uses `update_confirmed_mode()` to prevent redundant commands.
- **mavros_interface.py & constants.py:**
 - **MavrosInterface:** Executes `request_mode()` and `send_avoidance()` to dispatch repositioning commands.

- constants.py: Centralized configuration (COMMAND_RTL = 1, FAR_OFFSET = 43.0 m).

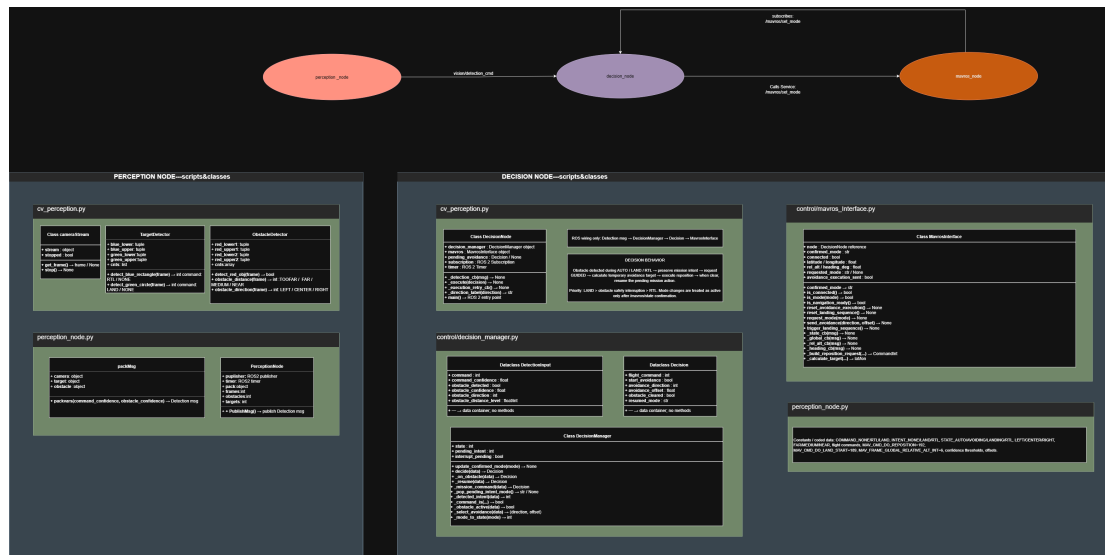


Figure 1: Object-Oriented Architecture and ROS 2 Node Interaction Diagram.

2 ROS 2 Node Structure, Topics, and MAVROS

2.1 Custom Interfaces & Node Architecture

The system consists of `perception_node` (Publisher) and `decision_node` (Subscriber). Communication relies on `Detection.msg` in the `pada_interfaces` package:

```
int8 command
float32 command_confidence
bool obstacle_detected
float32 obstacle_confidence
int8 obstacle_direction
int8 obstacle_distance
```

Listing 1: Custom ROS 2 Message Definition (`Detection.msg`)

2.2 MAVROS Integration

The Decision Node communicates with ArduPilot SITL using MAVROS:

- **Flight Mode Switches:** Calls `/mavros/set_mode` to command RTL, LAND, GUIDED, and AUTO.
- **Obstacle Avoidance:** Invokes `/mavros/cmd/command_int` with `MAV_CMD_DO_REPOSITION` (192), translating avoidance offsets into updated global GPS targets.

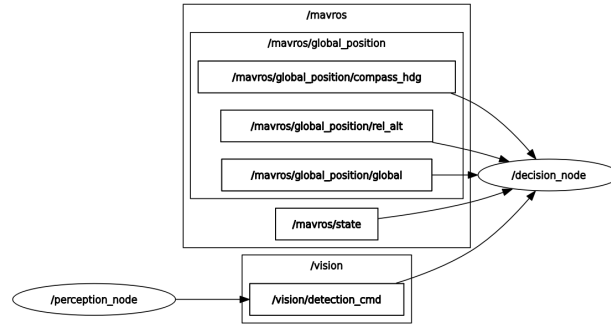


Figure 2: ROS 2 Node Computational Graph (rqt_graph).

3 Visual Perception and Mode Mapping

3.1 Perception Strategy & Shape Classification

Frames are converted to HSV via $Frame_{HSV} = \text{cv2.cvtColor}(Frame_{BGR}, HSV)$ for robust thresholding via $\text{cv2.inRange}()$. Contours are extracted (cv2.findContours) and approximated:

$$\epsilon = 0.02 \times P_{\text{contour}}, \quad \text{Vertices} = |\text{cv2.approxPolyDP}(\text{contour}, \epsilon, \text{True})| \quad (1)$$

4 Vertices: Quadrilateral (Rectangle). **> 8 Vertices:** Circle.

3.2 Mode Mapping Definition

Table 1: Visual Perception Command to Flight Mode Mapping

tableheader get Color	Tar- Geometry	Command Code	Triggered Flight Action
Blue	Rectangle (4 vertices)	COMMAND_RTL (1)	Return-To-Launch (RTL)
Green	Circle (> 8 vertices)	COMMAND_LAND (2)	Precision Landing (LAND)
Red	Any shape ($A > 500$ px)	Hazard Flag	Guided Obstacle Avoidance

4 Decision-Making and Obstacle Avoidance

4.1 Obstacle Distance & Direction Calculation

Distance relies on the area ratio: $A_{\text{ratio}} = (A_{\text{object}}/A_{\text{frame}}) \times 100\%$.

$$\text{Distance Level} = \begin{cases} \text{TOOFAR (2)}, & A_{\text{ratio}} < 5\% \\ \text{FAR (1)}, & 5\% \leq A_{\text{ratio}} < 10\% \\ \text{MID (0)}, & 10\% \leq A_{\text{ratio}} < 15\% \\ \text{NEAR (-1)}, & A_{\text{ratio}} \geq 15\% \end{cases} \quad (2)$$

Direction utilizes spatial moments ($c_x = M_{10}/M_{00}$) divided into three partitions: **LEFT (-1):** $0 \leq c_x < \frac{1}{3}W$, **CENTER (0):** $\frac{1}{3}W \leq c_x < \frac{2}{3}W$, **RIGHT (1):** $\frac{2}{3}W \leq c_x \leq W$.

4.2 Avoidance & Recovery Engine

When hazard data is received, the **DecisionManager** interrupts the mission for **GUIDED** mode. **MavrosInterface** calculates an offset GPS coordinate (e.g., +43.0 m) and dispatches the repositioning. Once the obstacle clears, it requests **AUTO** to seamlessly resume the waypoint mission.

5 Error Handling and Efficiency

- **Noise Filtering:** Background clutter is mitigated by strictly ignoring contours where $\text{Area} < 500$ px (if `cv2.contourArea(c) > 500:`).
- **Stream Reliability:** Latency crashes are prevented by verifying frame integrity (if `frame is None: return`) before processing.
- **Computational Efficiency:** To prevent processing lag, contours are sorted by area, and evaluation is restricted to the top 5 largest shapes (`[:5]`).

6 Evidence from the SITL Demonstration

A full mission validation was conducted in ArduPilot SITL. The aircraft executed an AUTO waypoint mission, encountered a simulated red obstacle requiring dynamic path alteration, and subsequently processed visual command targets to execute RTL and LAND mode transitions.

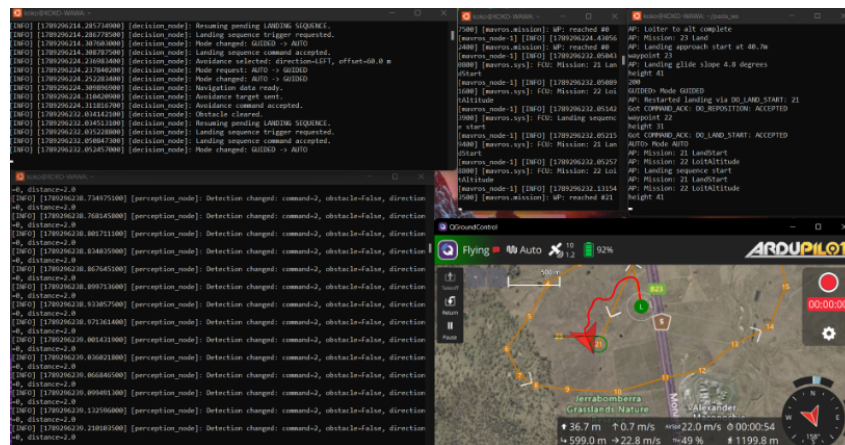
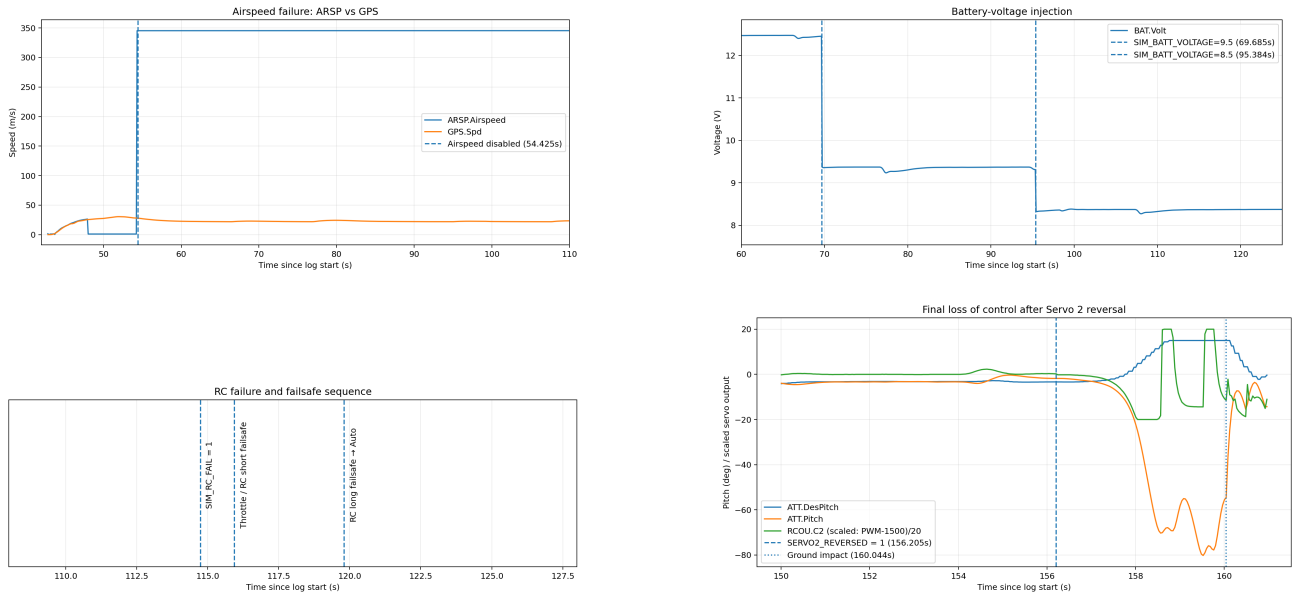


Figure 3: SITL flight execution and telemetry tracking.

Bonus Task 1 — SITL Log Debugging Challenge

The provided `BROKEN_LOGS.BIN` contains multiple deliberately injected faults. The analysis below distinguishes handled non-critical anomalies from the primary cause of control loss.

tableheader Find- ing	Time	Evidence / Interpretation
Airspeed Anomaly	48.005–155.025 s	<code>SIM_ARSPD_FAIL</code> produces abnormal airspeed (~ 345 m/s). The failure is detected/disabled by ArduPilot and navigation falls back to GPS; not the final crash cause.
RC Failure	114.745 s	<code>SIM_RC_FAIL=1</code> causes throttle failsafe. The aircraft safely switches to Auto at 119.805 s, demonstrating proper protective response.
Battery Drop	69.685 / 95.384 s	<code>SIM_BATT_VOLTAGE</code> reduced to 9.5 V and 8.5 V. No battery-failsafe action is recorded; does not explain the loss of control.
Elevator Reversal (Root Cause)	156.205 s	<code>SERV02_REVERSED=1</code> reverses the elevator response (<code>SERV02_FUNCTION=19</code>). Measured pitch diverges rapidly to -80° before impact at 27.48 m/s.



Flight-log evidence plots demonstrating sensor anomalies and pitch divergence.

Corrective Actions

1. Restore `SERV02_REVERSED = 0` to correct elevator deflection direction, and verify `SERV02_FUNCTION = 19` in SITL pre-flight checks.
2. Reset `SIM_RC_FAIL = 0` and verify the intended RC-failsafe settings.
3. Reset `SIM_ARSPD_FAIL = 0` and `SIM_ARSPD_FAILP = 0` to restore primary airspeed sensor fusion.
4. Restore nominal battery voltage telemetry and configure appropriate low-voltage failsafes.

Final Software Assessment

The integrated system supports all required operations: normal **AUTO** waypoint flight, **GUIDED** avoidance and mission recovery, and **RTL/LAND** visual command processing. The architecture remains modular, relying on safety mechanisms such as 10-frame visual confirmation, 20%/5% obstacle hysteresis, and pending-intent preservation.